Abdelmalek Essaadi University

Faculty of Sciences and Techniques of Tangier

Master's in IT Security and Big Data

Segmented Privileged Access Gateway

A Hands-On Lab for Network Segmentation,
Policy Enforcement & Session Brokering

Author:

Anas Ahrarache

Tools: GNS3 – VMware Workstation – FortiGate – JumpServer

https://github.com/AnasAhrarache/segmented-privileged-access-gateway

Contents

Executive Summary

This project designs, builds, and verifies a network architecture in which privileged access to a protected server is routed through a single, enforced, credential-vaulting broker – not as a policy convention, but as a physical impossibility to bypass at the network layer.

A FortiGate firewall segments three isolated network zones – administration, broker, and server – such that no policy exists permitting direct traffic between the administration zone and the server. The only path between them runs through JumpServer, an open-source Privileged Access Management (PAM) platform, which vaults the server's real credentials, brokers the SSH session on the operator's behalf, and records the entire session for audit.

Every architectural claim in this report is backed by a real test: the deny-by-default baseline was verified before any policy existed, an explicit allow policy was created and tested, then removed and re-verified as denied, and a full end-to-end session was established and recorded without the operator ever seeing the target server's real password.

Repository reference: Full configuration, screenshots, and source files are available in the public repository.
https://github.com/AnasAhrarache/segmented-privileged-access-gateway

1. Why This Architecture Matters

1.1 The Problem This Design Solves

In most small and mid-sized environments, administrators connect directly to servers using credentials they know personally. This creates three concrete, well-documented risks:

Risk	Description	How This Architecture Mitigates It
Lateral movement	A compromised administration workstation can pivot directly to sensitive servers if no network barrier exists.	FortiGate enforces deny-by-default between the administration zone and the server zone; no route exists except through the broker.
No audit trail	Direct SSH access with a shared or personally-known credential leaves no reliable record of who did what, and when.	JumpServer records every brokered session in full (keystrokes, commands, and video replay), with a separate login/operation audit log.
Credential sprawl	Every administrator who needs server access must know the real password, multiplying the number of places a credential can leak.	The server's real credential is vaulted inside JumpServer and never exposed to the operator; access is granted through explicit authorization, not credential possession.

Table 1.1: Business risks addressed by the architecture

1.2 What This Lab Is (and Is Not)

In Scope – Implemented and Verified	Out of Scope – Not Implemented
Deny-by-default network segmentation	Centralized identity provider (IdP)
Explicit, tested policy enforcement	Continuous identity / context verification
Least privilege at the service level	Endpoint posture checks
Privileged credential vaulting	Dynamic / adaptive policy engine
Separation of duties (admin vs. operator)	Microsegmentation at the individual workload level
Full session recording and audit trail	Full organization-wide Zero Trust architecture

Table 1.2: Explicit scope boundary, stated upfront rather than left implicit

This project applies specific, verifiable Zero Trust principles to the privileged-access perimeter. It is not, and does not claim to be, a complete Zero Trust architecture at the organizational level. Section 8.3 revisits this distinction in detail with a full checklist.

2. Lab Architecture

2.1 Components

Tool	Role
GNS3	Network topology orchestration and node interconnection
VMware Workstation Pro	Hypervisor hosting all virtual machines
FortiGate (VM64)	Firewall enforcing segmentation and access policies
Ubuntu Desktop	Administration workstation (AdminVM)
Ubuntu Server	Protected target server (Server_VM) and PAM broker host (Broker_VM)
JumpServer (v4.10)	Privileged Access Management (PAM) platform

Table 2.1: Lab components

2.2 Logical Architecture

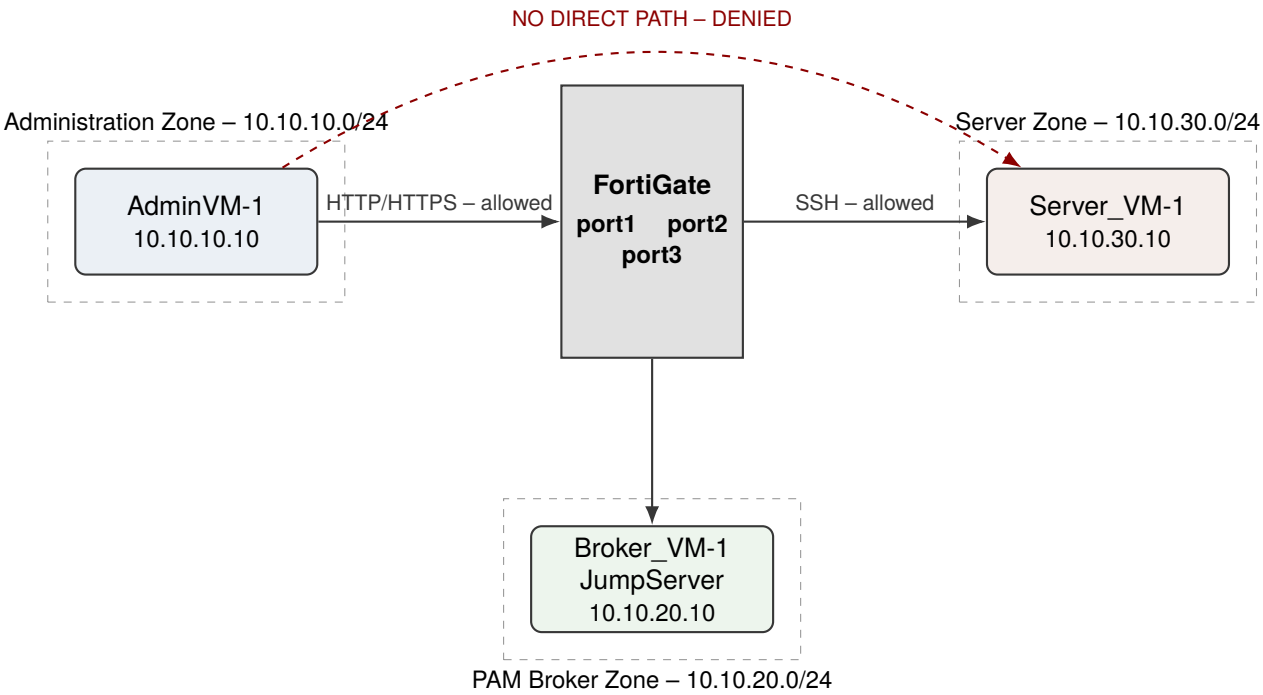

Figure 2.1: Logical architecture: FortiGate as the single point of passage between the three zones. No direct path exists between administration and the server.

2.3 Addressing Plan

Zone	FortiGate Interface	Subnet	Host
Administration	port1	10.10.10.0/24	AdminVM-1 (10.10.10.10)
PAM Broker	port3	10.10.20.0/24	Broker_VM-1 – JumpServer (10.10.20.10)
Server	port2	10.10.30.0/24	Server_VM-1 (10.10.30.10)

Table 2.2: Addressing plan for the lab's three zones

This three-zone division gives the firewall an independent observation point for each actor – administrator, broker, and server – and allows distinct policies per flow, rather than relying on per-host rules within a shared segment.

2.4 Technical Topology in GNS3

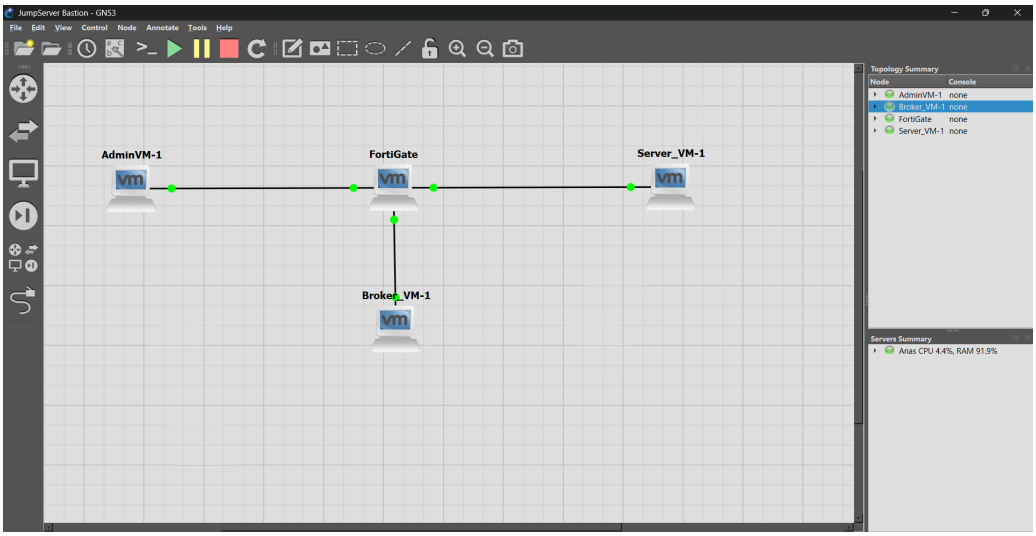

Figure 2.2: Lab topology in GNS3: AdminVM-1, FortiGate, Broker_VM-1, and Server_VM-1

Repository reference: The raw GNS3 project files and VM export notes are documented in `configs/`.
https://github.com/AnasAhrarache/segmented-privileged-access-gateway

3. Network Segmentation and Firewall Policies

3.1 Baseline: Deny by Default

FortiGate applies an implicit deny policy by default between any pair of interfaces with no explicit policy defined. This was verified before any rule was created, establishing a secure baseline: no traffic was permitted between any of the three zones.

3.2 Validating Explicit Enforcement

To confirm the firewall enforces configured rules rather than an implicit allowance, a temporary policy was created between port1 (administration) and port2 (server), tested, and then removed.

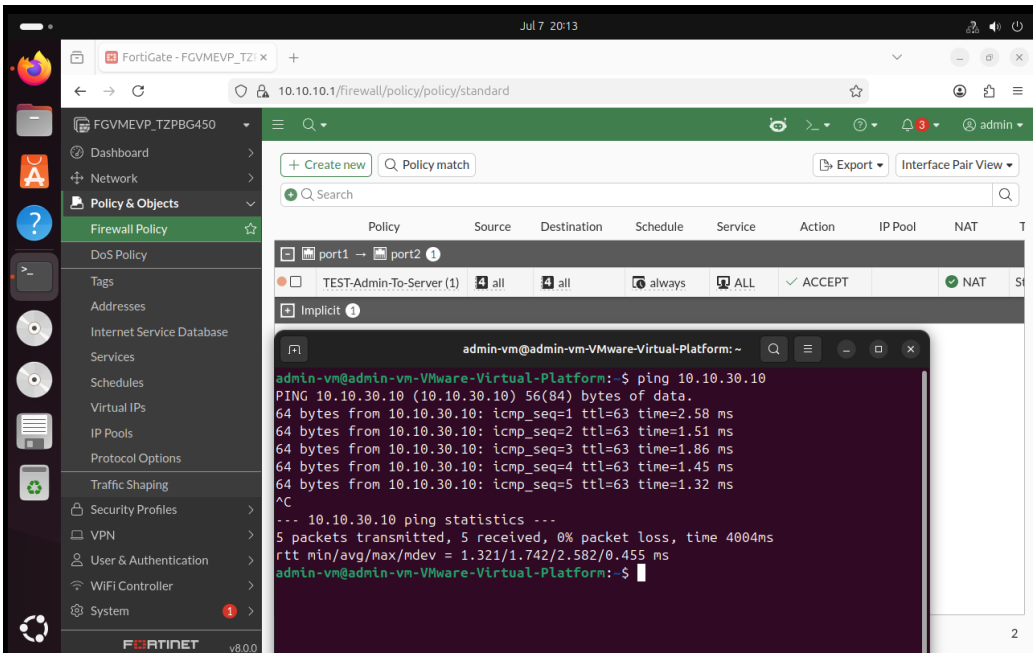

Figure 3.1: Temporary validation: explicit port1-to-port2 authorization, tested then revoked

This confirmed three points: without a policy, communication fails consistently (0% success); with an explicit allow policy, communication succeeds immediately (0% packet loss); after the policy is removed, communication fails again – proving behavior is governed entirely by active configuration, not residual state.

3.3 Final Policies

Policy	Source → Destination	Service	Action
Admin-To-Broker	port1 → port3	HTTP / HTTPS	ACCEPT
Broker-To-Server	port3 → port2	SSH	ACCEPT
(implicit)	port1 → port2	–	DENY

Table 3.1: Firewall policies applied in the final architecture

No policy exists between port1 and port2. This deliberate absence is what guarantees the only path between the administrator and the server runs through the broker. Services are restricted to the strict minimum required per flow, applying least privilege at the service level in addition to zone-based segmentation.

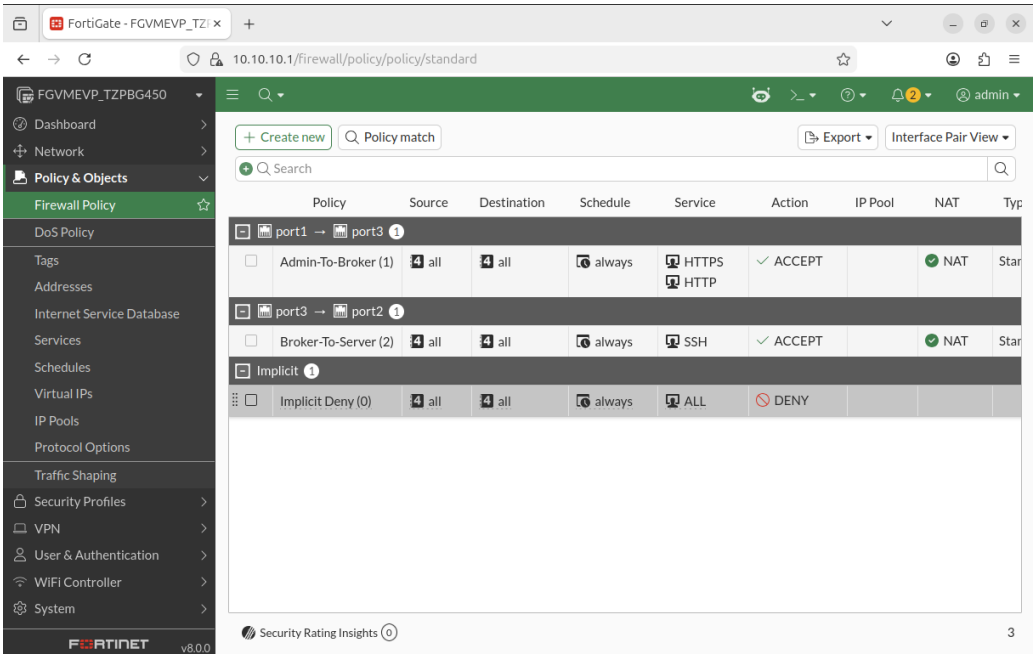

Figure 3.2: Final firewall policies in the FortiGate interface

Repository reference: Every CLI command and GUI field used to build these policies is recorded in `configs/fortigate-policies.md`.
https://github.com/AnasAhrarache/segmented-privileged-access-gateway

4. Deploying the PAM Broker

4.1 Broker Selection

JumpServer, an open-source PAM platform, was selected to build and validate this lab. It was chosen pragmatically: it shares the same core architectural principles as commercial platforms such as Wallix Bastion – credential vaulting, session brokering, recording, and granular authorization – and its deployment could be completed within the project timeline. The network segmentation and firewall policies designed in this project are independent of the specific PAM product deployed; substituting a different broker changes only the configuration inside the broker zone, not the surrounding architecture (see Section 9.2).

4.2 Installation

JumpServer was deployed on a dedicated Ubuntu Server virtual machine (Broker_VM) using the official Docker-based installer:

```
cd /opt
curl -sSL https://github.com/jumpserver/jumpserver/releases/latest/download/quick_start.sh |
    sudo bash
```

Since installation requires internet access to pull Docker images, the machine was temporarily attached to a NAT network during setup, then reintegrated into its isolated 10.10.20.0/24 segment.

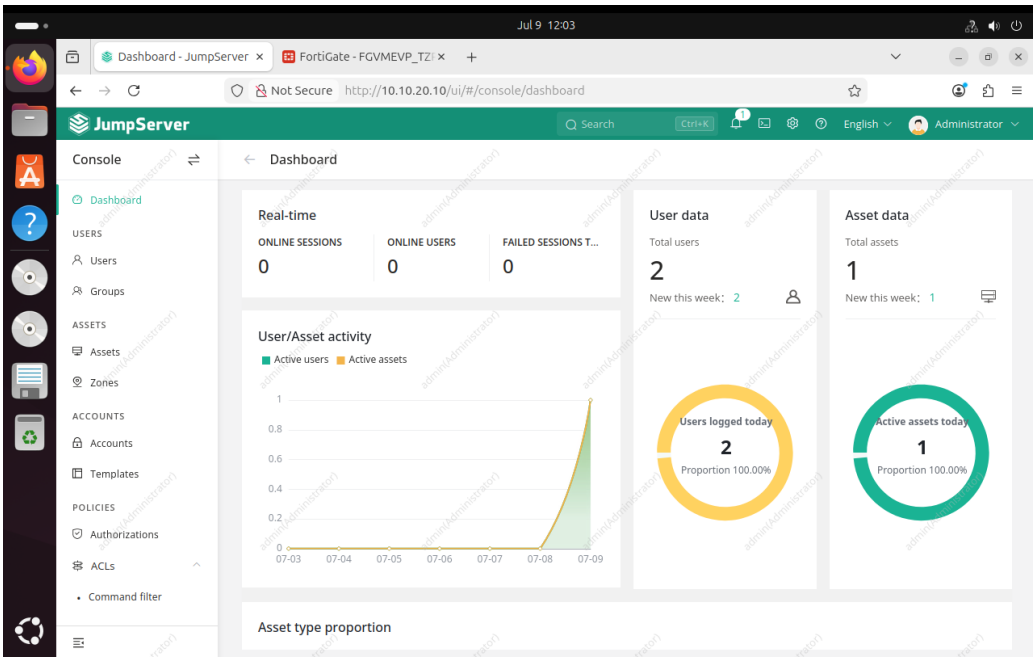

Figure 4.1: Dashboard of the deployed JumpServer instance

4.3 Registering the Target Server (Asset)

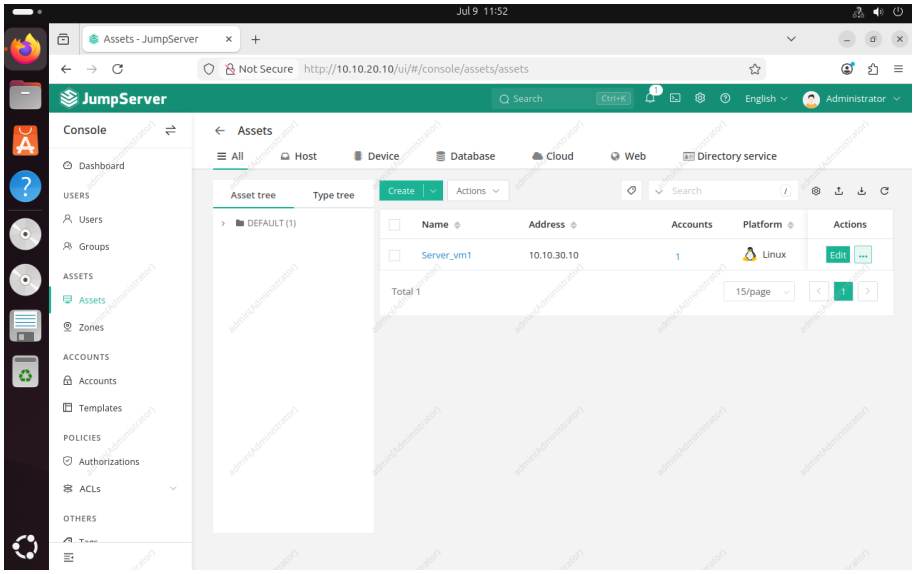

Figure 4.2: Registration of Server_VM1 as an Asset in JumpServer

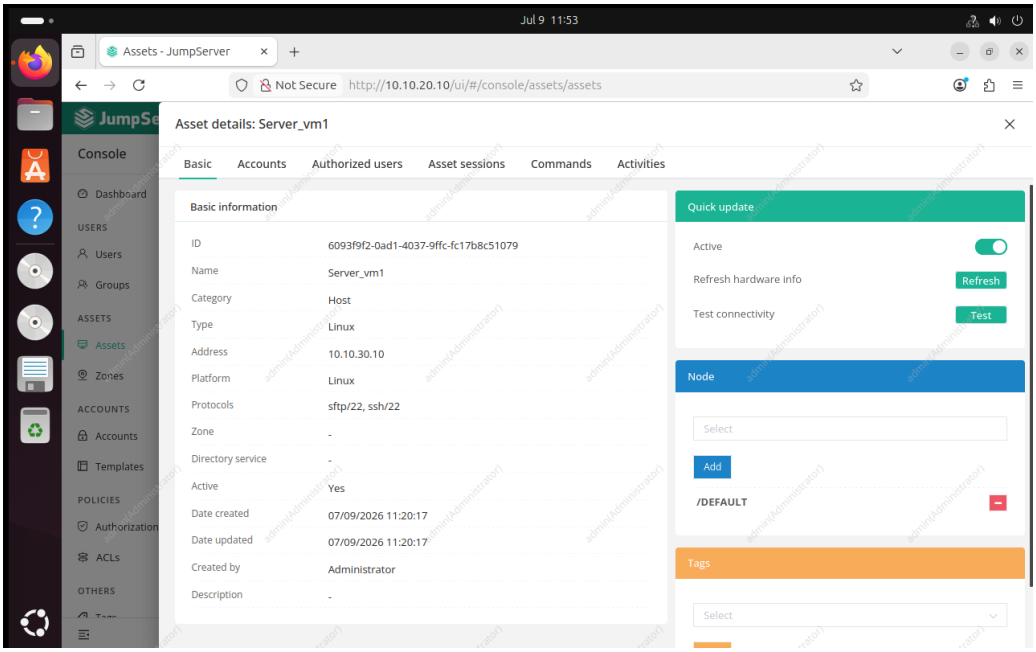

Figure 4.3: Details of the Server_VM1 Asset: protocols, zone, status

4.4 Credential Vaulting (Account)

A technical account linked to this Asset stores the server's real SSH credentials. This account is the password vault: its secret is never communicated directly to the end user.

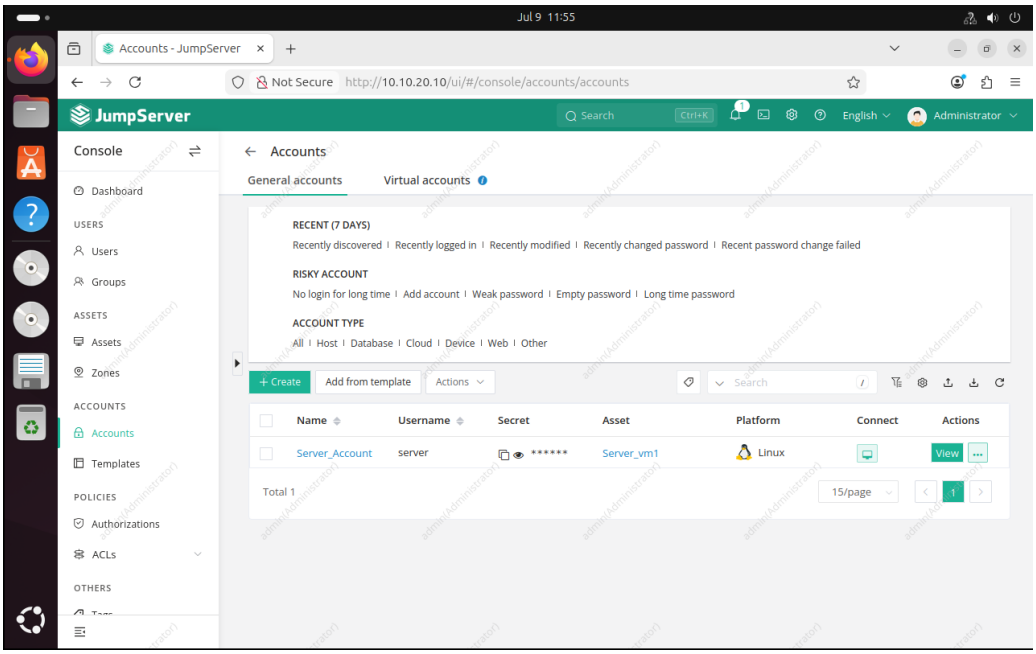

Figure 4.4: Technical account Server_Account linked to the Server_VM1 Asset

4.5 Separation of Duties

A dedicated operational user, `Operator1`, was created separately from the platform's `Administrator` account – the account configuring the PAM system is distinct from the account authorized to use it day to day.

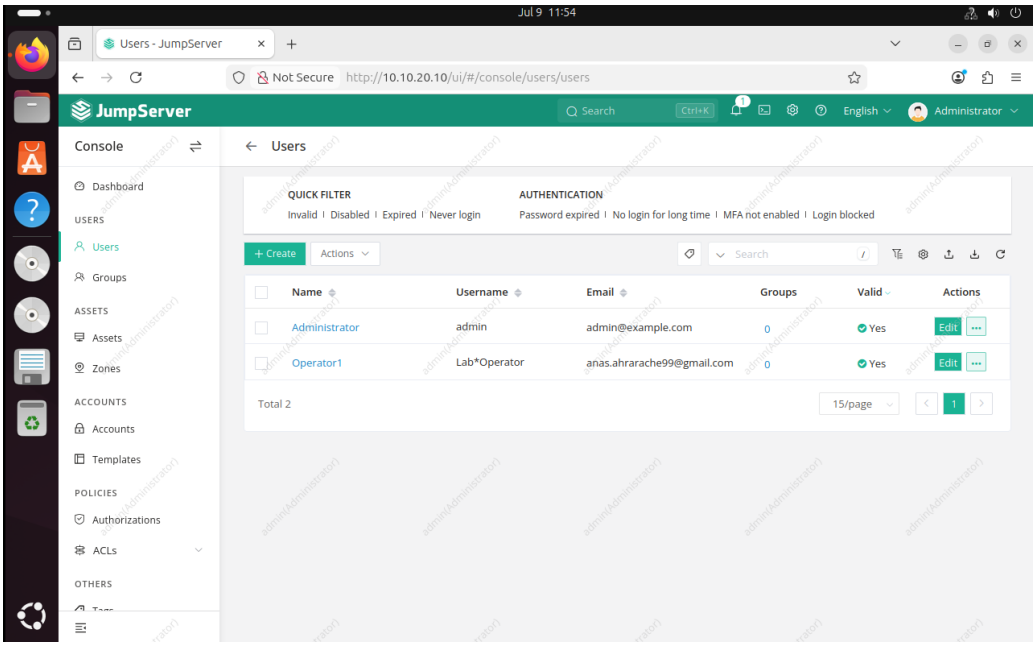

Figure 4.5: JumpServer users: Administrator and Operator1

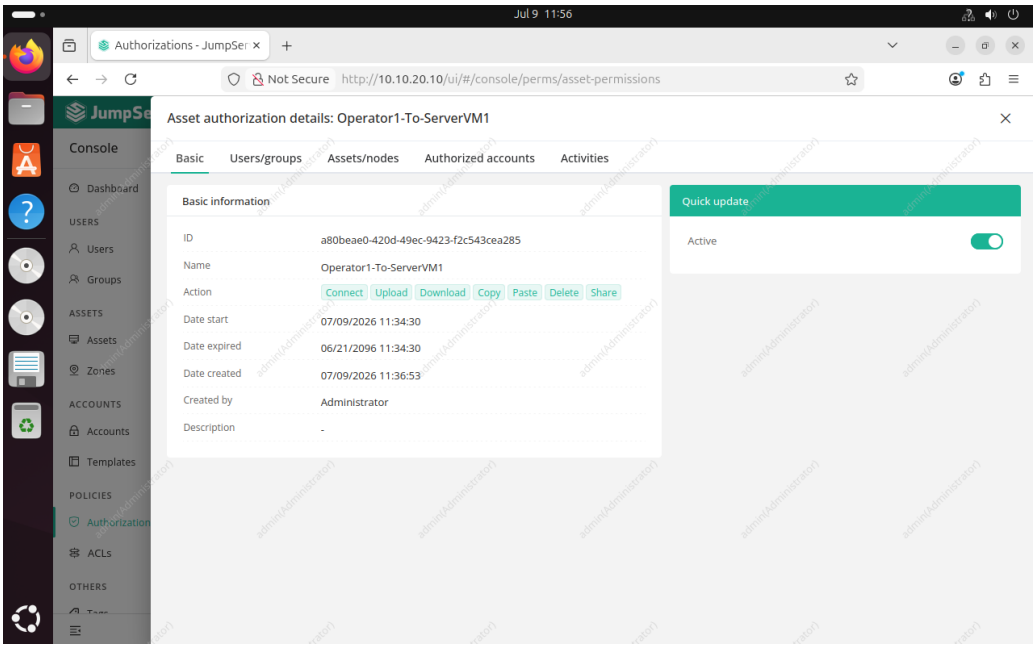

Figure 4.6: Authorization rule Operator1-To-ServerVM1

5. Connection Test and Session Brokering

5.1 Functional Flow

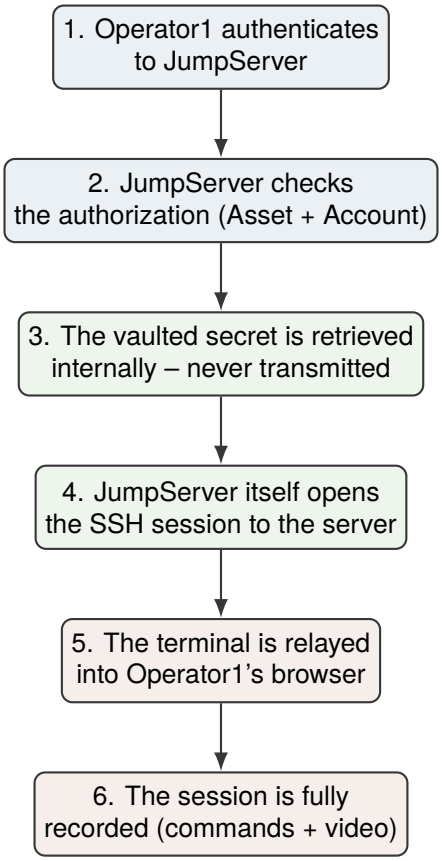

Figure 5.1: Functional sequence of privileged access brokering via JumpServer

5.2 Workbench and Connection

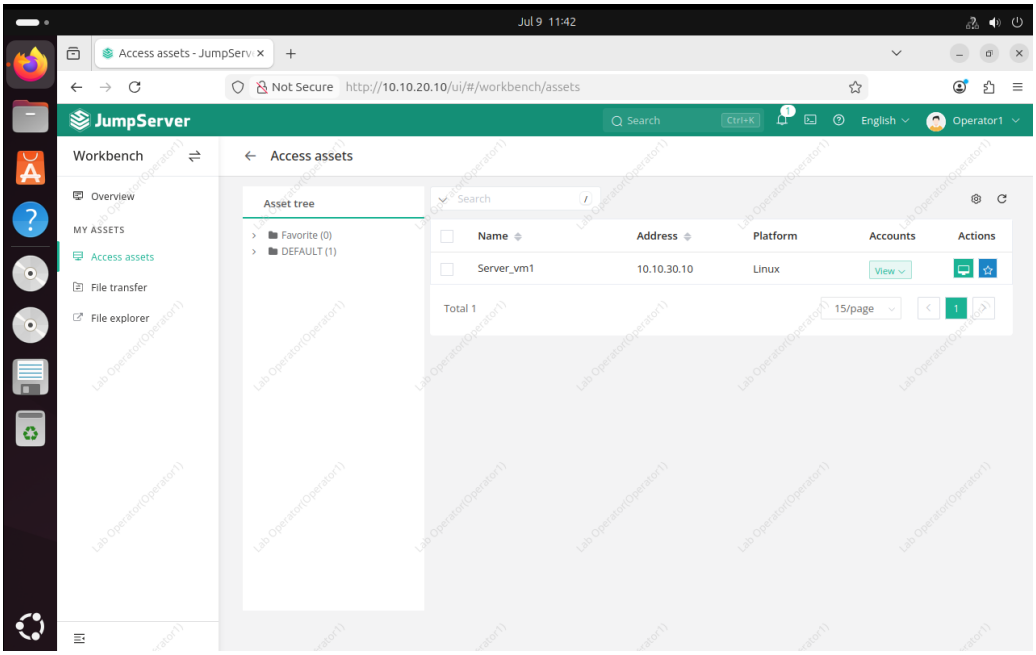

Figure 5.2: Operator1's workbench: access limited to Server_VM1

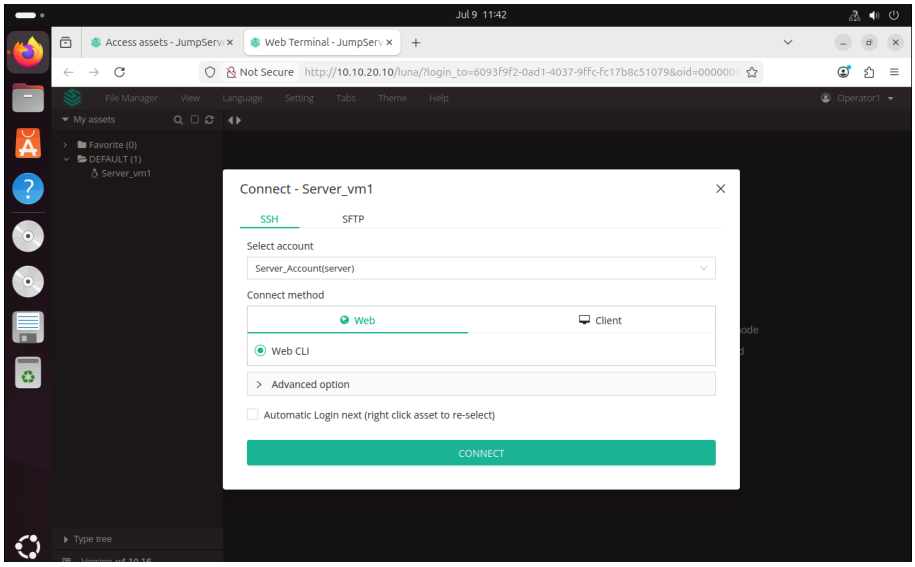

Figure 5.3: Connection window: account and method selection (Web CLI)

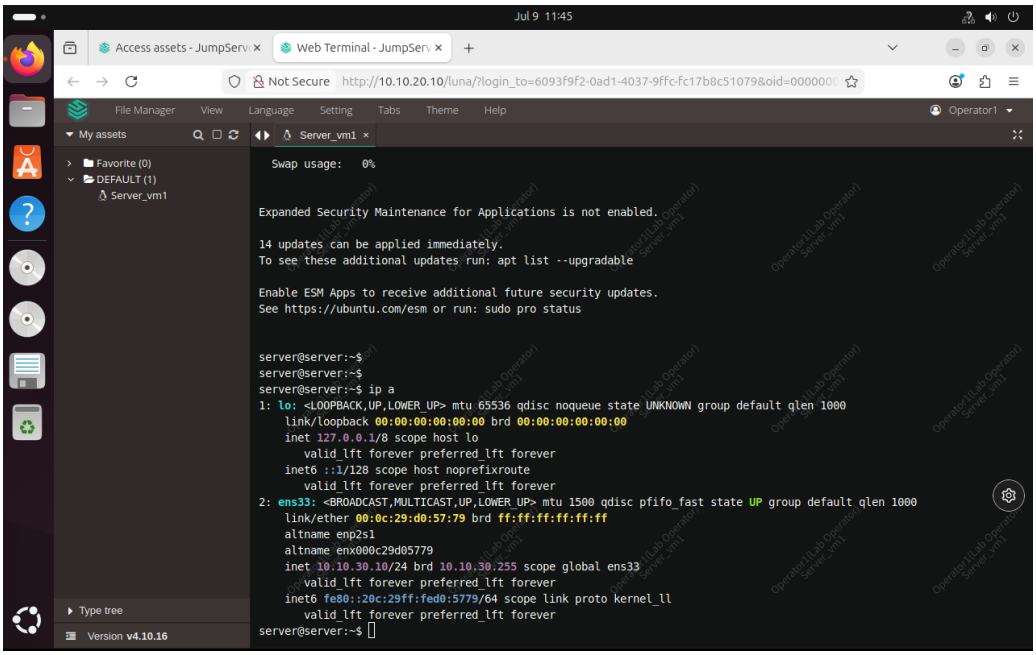

Figure 5.4: Active web terminal: SSH session established to Server_VM1 via JumpServer

6. Audit, Traceability, and Session Recording

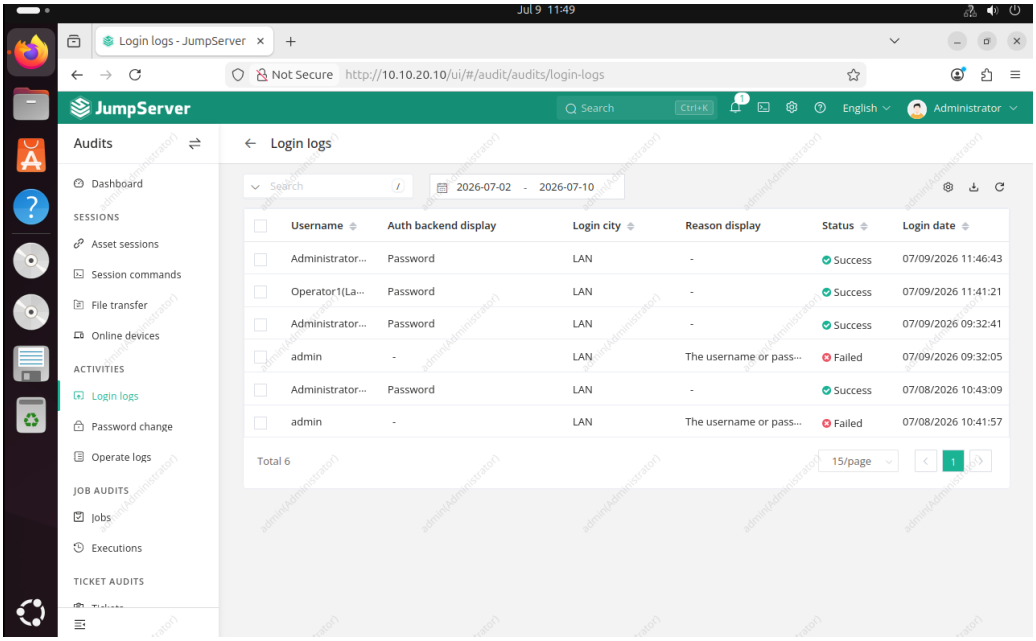

Figure 6.1: JumpServer login logs

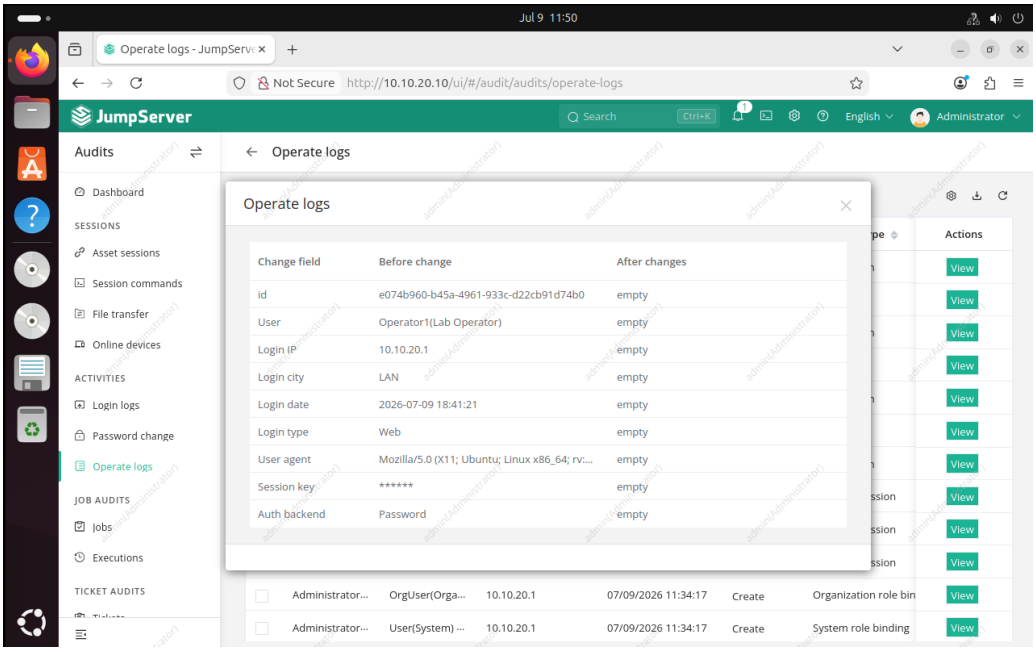

Figure 6.2: Detail of an operation log entry

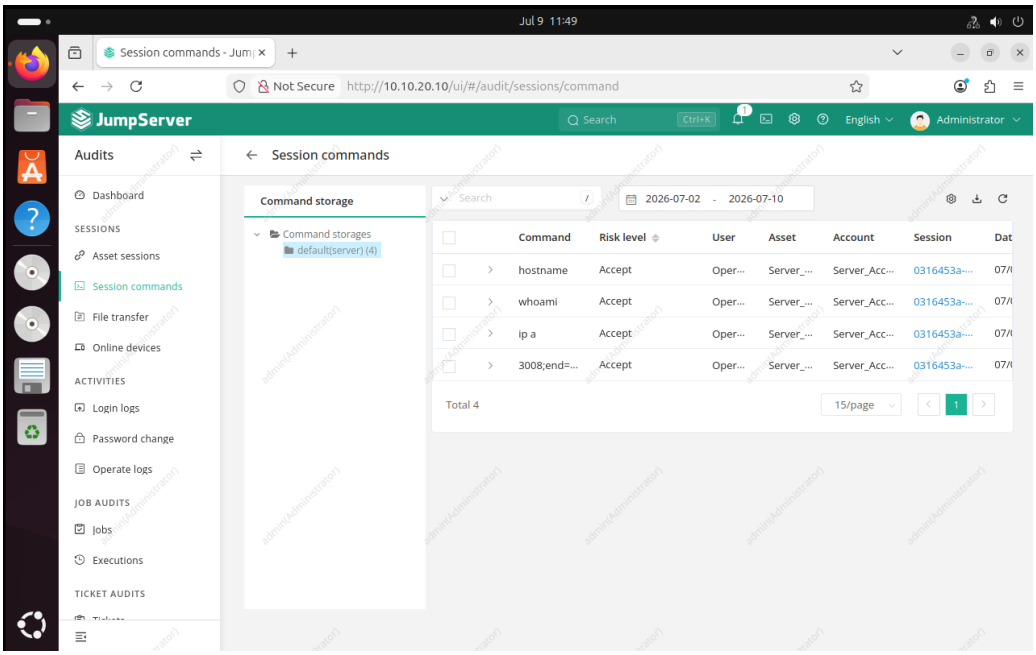

Figure 6.3: Commands entered during the session, with associated risk level

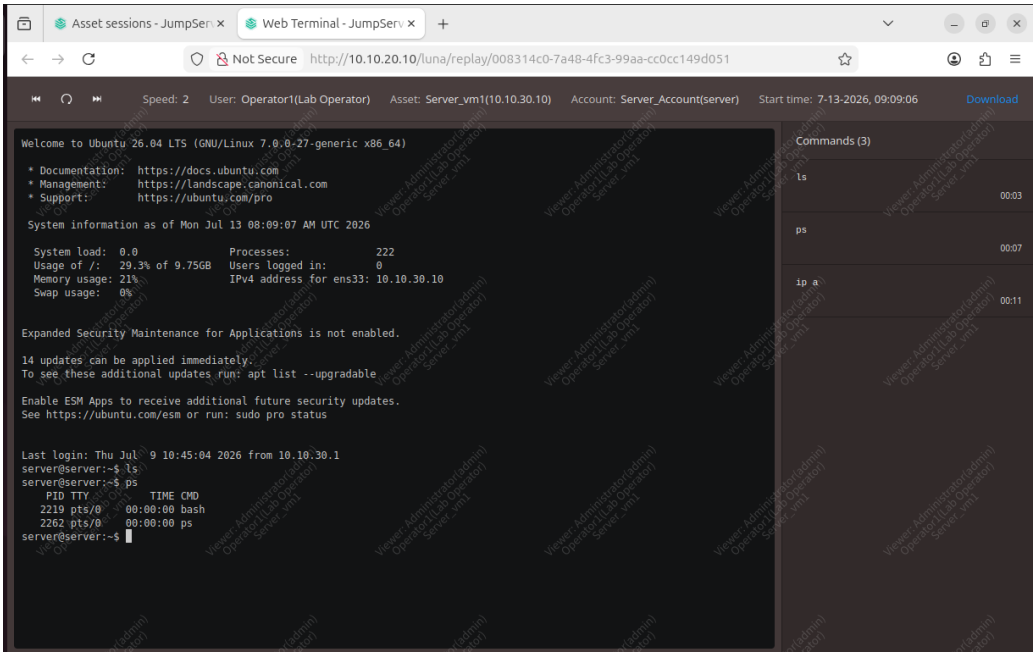

Figure 6.4: Replay of the recorded session: terminal, user, account used, and commands

7. Verification and Test Results

7.1 Test Matrix

Every architectural claim in this report corresponds to a test that was actually executed, not assumed. The table below summarizes all nine test cases performed during the build.

#	Flow	Condition	Expected	Result
T1	Admin → Broker	No policy exists (baseline)	Deny	**PASS**
T2	Broker → Server	No policy exists (baseline)	Deny	**PASS**
T3	Admin → Server	No policy exists (baseline)	Deny	**PASS**
T4	Admin → Server	Temporary allow policy created	Allow	**PASS**
T5	Admin → Server	Temporary policy removed	Deny	**PASS**
T6	Admin → Broker	Final policy (HTTP/HTTPS)	Allow	**PASS**
T7	Broker → Server	Final policy (SSH)	Allow	**PASS**
T8	Admin → Server	Final state – no policy exists	Deny	**PASS**
T9	Operator1 → Server (via Broker)	End-to-end brokered SSH session	Success	**PASS**

Table 7.1: Full verification test matrix – 9/9 tests passed. T9: credential never exposed to the operator and the full session was recorded (see Ch. 6).

7.2 Key Evidence

Five figures in this report carry particular evidentiary weight, each proving a distinct architectural claim:

- **Figure 3.2 (GNS3 topology)** – proves the three-zone segmented topology was actually built, not just diagrammed.
- **Figure 4.1 (temporary policy test)** – proves explicit enforcement: deny, then allow, then deny again, on command.
- **Figure 4.2 (final firewall policies)** – proves the production policy set matches the least-privilege design.
- **Figure 6.3 (SSH session established)** – proves credential-less, brokered connection actually works end to end.
- **Figure 7.4 (session replay)** – proves full session auditability, not just a claim of "logging enabled."

Repository reference: All screenshots referenced above, at full resolution, are available in `screenshots/`.
https://github.com/AnasAhrarache/segmented-privileged-access-gateway

8. Security Analysis and Threat Model

8.1 What This Architecture Protects Against

Threat	Mitigation	Status
Lateral movement from a compromised admin workstation to the server	Network segmentation, deny-by-default, single enforced choke point (FortiGate)	Verified (T3, T8)
Credential exposure to the operator	Credential vaulting in JumpServer; real secret never transmitted to the user	Verified (T9)
Untraceable privileged access	Full session recording, per-command audit, login/operation logs	Verified (Ch. 6)
Unauthorized use of the PAM broker by an unrelated account	Explicit per-asset authorization; separation of duties (Administrator vs. Operator1)	Verified
Unauthorized firewall policy tampering	Partially addressed – see residual risks below	Partial

Table 8.1: Threats addressed by the architecture, with verification status

8.2 Residual Risks and Limitations

Risk	Current State	Mitigation Strategy
FortiGate admin interface reachable from the administration zone	port1 allows HTTPS/SSH to FortiGate itself	Dedicated out-of-band management network, RBAC on the firewall, MFA on the super-admin account
Compromised endpoint reusing a legitimate broker session	Segmentation blocks direct bypass, not legitimate-session misuse	MFA on JumpServer accounts, short session TTL, Command ACL filtering
Zone-level rather than host-level policy granularity	Policies scoped per interface/-zone, not per host	FortiGate address objects scoped to individual hosts
No centralized identity provider or continuous verification	Static, one-time authentication only	Integrate an IdP (e.g., Entra ID, Okta) with conditional access
No automated anomaly detection	Logs exist but are not actively correlated	Feed FortiGate and JumpServer logs into a SIEM (e.g., Wazuh, ELK)

Table 8.2: Known residual risks and their mitigation strategy

8.3 Honest Scope Assessment

Zero Trust Principle	Implemented?
Deny-by-default	Yes
Explicit, tested enforcement	Yes
Least privilege (service-level)	Yes
Separation of duties	Yes
Centralized identity provider (IdP)	No
Continuous identity / context verification	No
Endpoint posture checks	No
Dynamic / adaptive policy engine	No
Microsegmentation at the individual workload level	No (zone-level only)

Table 8.3: Explicit checklist – what is and is not implemented relative to a full Zero Trust model

This project applies targeted, verifiable Zero Trust principles to the privileged-access perimeter. It does not constitute a complete, organization-wide Zero Trust architecture, and does not claim to.

9. Evolution Roadmap

9.1 Improvement Tiers

Improvement	Tier	Effort
Restrict policies to specific address objects per host	Immediate	Low
Enable multi-factor authentication on JumpServer	Immediate	Low
Configure Command ACL rules for risky commands	Short-term	Medium
Move FortiGate management to an out-of-band network	Short-term	Medium
Correlate FortiGate and JumpServer logs in a SIEM	Short-term	Medium
Add a second target server with a distinct access tier	Long-term	Medium
Integrate a centralized identity provider (IdP)	Long-term	High
Substitute JumpServer with Wallix Bastion	Long-term	Depends on licensing

Table 9.1: Evolution roadmap, grouped by priority and estimated effort

9.2 Product-Agnostic Architecture

A deliberate design goal of this project was to keep the network architecture independent of the specific PAM broker deployed inside it.

Stays the Same	Changes
Three-zone network segmentation	Broker VM operating system / software stack
FortiGate interfaces and IP addressing	Asset, account, and user configuration inside the broker
Firewall policies (Admin-To-Broker, Broker-To-Server)	Session recording mechanism and audit log format
The deny-by-default choke point design	Broker-specific hardening (MFA, session TTL, Command ACL)

Table 9.2: What changes and what remains constant if the PAM broker is replaced

This answers, in advance, a question any reviewer would reasonably ask: replacing JumpServer with Wallix Bastion, or any other PAM platform, requires no change to the network segmentation or FortiGate policies documented in Chapter 3 – only the broker's internal configuration would be redone.

10. Conclusion

For a security team hiring a junior engineer, this project signals three things:

1. **The ability to design and verify controls, not just configure them.** Every claim in this report – from the deny-by-default baseline to the final brokered session – was tested, not assumed. Nine test cases were executed and documented, with real packet-loss figures and session captures as evidence.

2. **An honest, precise understanding of security scope.** Rather than overclaiming a full Zero Trust architecture, this report explicitly separates what was implemented and verified from what would be required to extend it – and names the residual risks that remain, along with concrete mitigation strategies.

3. **A clear, product-agnostic evolution path.** The architecture was deliberately built so that the underlying PAM broker can be swapped without touching the network design, and a tiered roadmap identifies exactly what to build next, in what order, and at what cost.

This lab is a solid, reproducible, and fully documented foundation – one built to be extended, not just demonstrated.

Repository reference: Full source, configuration files, and the complete screenshot set are available at the repository below.
https://github.com/AnasAhrarache/segmented-privileged-access-gateway

A. Repository Structure

```
segmented-privileged-access-gateway/
 README.md
 report/
    report_english.pdf
    rapport.pdf # French version
 configs/
    fortigate-policies.md # Full FortiGate CLI/GUI configuration trail
 screenshots/
     architecture/
     fortigate-policies/
     jumpserver/
```

https://github.com/AnasAhrarache/segmented-privileged-access-gateway